

Design Responsivo

Design Responsivo é uma forma de garantir que o site “responda” ao tamanho da tela do usuário, seja um monitor, tablet ou um celular.

Meta Tag Viewport

Para criar um design responsivo é necessário garantir que o site “responda” a qualquer tamanho de tela, então precisamos adicionar um comando na tag <head> para que o navegador identifique que o objetivo é utilizar como fonte primária a tela do dispositivo do usuário, caso contrário o navegador vai tentar simular a tela do desktop no celular, para isso utilizamos a tag a baixo:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Unidades Relativas vs Fixas

Uma boa pratica é não utilizar o “px” como unidade de tamanho pois ele é uma unidade fixa e não vai se adaptar a tela do dispositivo, temos 3 tipos de unidade unidades relativas:

% - Percentual em relação ao pai

vw/vh - Percentual da largura ou altura da tela(viewport)

em/rem - Baseado no tamanho da fonte (ótimo para acessibilidade)

Media Queries (Onde se identifica a responsividade)

As medias queries permitem aplicar estilos apenas em certas condições como por exemplo a largura da tela, aqui temos um exemplo onde o fundo fica azul em telas grandes e verde em telas menores de 600px.

```
body {  
  background-color: blue;  
}  
  
@media (max-width: 600px) {  
  body {  
    background-color: green;  
  }  
}
```

Estruturas modernas: FlexBox e Grid

Hoje em dia, raramente usamos Media Queries para consertar tudo no design, usamos sistemas de layout que já nascem flexíveis, podemos utilizar o flex-box com o flex-wrap: no-wrap que faz com que os itens caibam na tela e caso necessário ele joga para baixo o próximo item, mesmo que seja uma flex-direction: row, também temos o atributo flex que é utilizado para definir como eles devem crescer, encolher e qual será seu tamanho base, assim ele ajusta automaticamente o tamanho do elemento ao espaço disponível.

Aqui temos um exemplo de um card que ocupa a linha toda no mobile, porém fica lado a lado no desktop

```
<div class="card-container">
  <div class="card">Item 1</div>
  <div class="card">Item 2</div>
  <div class="card">Item 3</div>
</div>

<style>
  .card-container {
    display: flex;
    flex-wrap: wrap; /* Permite que os itens "pulem" de linha */
    gap: 1rem;
  }

  .card {
    flex: 1 1 300px; /* Cresce, encolhe e tem base de 300px */
    background: #f4f4f4;
    padding: 2rem;
    text-align: center;
    border: 1px solid #ddd;
  }
</style>
```

Aqui também temos um exemplo de Grid onde criamos um container utilizando a propriedade auto-fit que permite que o navegador decida quantos itens cabem por linha sozinho sem a necessidade de informar a quantidade colunas.

```
.container {
  display: grid;
  /* Cria colunas de no mínimo 250px, distribuindo o espaço restante */
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

Mobile-first e Desktop-first

A diferença de um para outro é simples, o mobile-first começa o design pelos menores dispositivos com isso utilizamos no médio querie o min-width onde ele verifica a largura mínima para aplicar as novas regras de estilo, já o desktop first ele utiliza na Media queries o max-width que é a largura máxima para aplicar os estilos.

Além da forma de aplicar uma das diferenças é que o foco do mobile-first tem foco em performance e simplicidade, enquanto o desktop-first utiliza um visual mais rico e complexo, em relação ao código o MF adiciona novos estilos conforme a tela cresce o DF sobrescreve/remove estilos conforme a tela diminui.

Tendo em vista todas as informações o mobile-first é considerado por muitos o melhor método de utilização, ele tem uma economia de dados para dispositivos mobile já que ele baixa o CSS básico primeiro e não precisa processar os estilos pesados do desktop para depois ignorá-los. Também podemos dizer que é mais fácil organizar o conteúdo essencial em uma tela pequena e depois espalhar para o desktop do que tentar colocar um site gigante dentro de um mobile.

Boas Práticas

- Utilização do mobile-first: comece estilizando para um celular fora da media query e depois utiliza o min-width para adicionar complexidade para telas maiores.
- Imagens Fluidas: Sempre defina um max-width: 100% e height: auto para que suas imagens não transbordem o container.
- Touch: Lembre que botões para o celular precisam de pelo menos 44x44px para serem clicáveis com o polegar.
- Não quebre o layout: Evite larguras fixas (800px), use max-width ou width em 100%.